

Suzaku Core Review

A smart contract assessment of the Suzaku Core contracts was performed by Octane Security, with a focus on the security aspects of the application's implementation.

About Octane Security

[Octane Security](#) is an AI-enhanced blockchain security firm dedicated to safeguarding digital assets. We combine security research with cutting-edge machine learning models to maximize protocol security and minimize risk. Our team ranks in the top of audit contest leaderboards, captures bug bounties, and has served big-name clients such as Alchemy, Optimism, Notional Finance, Blast, and Graph Protocol in the past.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We also rely on information that is provided by the client, its affiliates and partners for vulnerability identification. We can not guarantee the absense of vulnerabilities, issues, flaws, or defects after the review. It is up to the client to decide whether to implement recommendations and we take no liability for invalid recommendations. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Security Assessment Summary

The scope of this review was limited to files at commit: [8d74dbfcaae4dbfe994365b550fc9a758fa163e8](#)

Scope

The following smart contracts were in scope of the audit:

- `src/contracts/middleware/*`
- `src/contracts/vault/LSTWrapper*`
- `src/contracts/rewards/*`
- `src/contracts/VaultHelper.sol`
- `src/contracts/L1Registry.sol`

Findings

Finding Number	Finding Title
M-1	Manual stake updates allow operators to bypass the secondary collateral requirements
M-2	Sequential weight updates on the same validator cause stake accounting drift
I-1	Missing constructor validation in UptimeTracker
I-2	Silent vault skipping in <code>_calculateAndStoreVaultShares</code> hinders debugging
I-3	Duplicate code in LSTWrapper implementations
I-4	<code>VAULT_REMOVAL_EPOCH_DELAY</code> lacks validation despite suggesting fixed value
I-5	Magic number <code>1</code> used instead of <code>PRIMARY_ASSET_CLASS</code> constant
I-6	Unnecessary unchecked loop increments
I-7	<code>L1Registry.notZeroAddress</code> has overly specific parameter naming
I-8	Misleading variable name in <code>RewardsNativeToken.maxClaimableEpoch</code>
I-9	<code>LSTWrapper.mint()</code> missing <code>maxMint</code> bypass for owner's first mint
I-10	Unused <code>rewardsToken</code> parameter in <code>VaultHelper</code> reward functions
I-11	Duplicate code in VaultHelper functions
I-12	Redundant <code>getAllOperators</code> call in <code>distributeRewards</code>

M-1 Manual stake updates allow operators to bypass the secondary collateral requirements

Description

An operator's stake can be updated in two ways: manually by the operator themselves via `initializeValidatorStakeUpdate`, or permissionlessly via `forceUpdateNodes` during the final `UPDATE_WINDOW` of each epoch.

The `forceUpdateNodes` function brings an operator's nodes into compliance with system parameters, including the minimum secondary collateral requirement. It checks this requirement and reduces stake from nodes until compliance is achieved, but skips any nodes that have a pending weight update:

```
function forceUpdateNodes(address operator, uint256 limitStake) external {
    // ...
    bool secondaryOk = _requireMinSecondaryCollateralClasses(0, operator);
    // ...
    for (uint256 i = length; i > 0 && (leftoverStake > 0 || !secondaryOk);) {
        i--;
        bytes32 nodeId = nodesArr[i];
        bytes32 valID = _vid(nodeId);
        if (balancerValidatorManager.isValidatorPendingWeightUpdate(valID)) {
            continue; // Skip nodes with pending updates
        }
        // ... reduce stake or end validation
    }
}
```

However, `initializeValidatorStakeUpdate` does not check the secondary collateral requirement:

```
function initializeValidatorStakeUpdate(bytes32 nodeId, uint256 stakeAmount)
external {
    _updateGlobalNodeStakeOncePerEpoch();
    if (!operatorNodes[msg.sender].contains(nodeId)) {
        revert AvalancheL1Middleware__NodeNotFound(nodeId);
    }
    // ... stake validation ...
    _initializeValidatorStakeUpdate(msg.sender, validationID, stakeAmount);
    // No secondary collateral check here
}
```

This creates an exploit path:

1. Operator registers with sufficient secondary collateral to pass the initial check in `addNode`
2. Operator withdraws or reduces their secondary collateral
3. Before `forceUpdateNodes` can be called in the update window, operator calls `initializeValidatorStakeUpdate` with a minimal stake change on each node
4. When `forceUpdateNodes` is called, it skips all nodes because `isValidatorPendingWeightUpdate` returns true for each
5. Operator repeats this each epoch, maintaining validators without meeting secondary collateral requirements

This allows operators to earn rewards on primary collateral without maintaining the required secondary collateral backing.

Recommendation

Add a secondary collateral check at the end of `initializeValidatorStakeUpdate`:

```
function initializeValidatorStakeUpdate(bytes32 nodeId, uint256 stakeAmount)
external {
    _updateGlobalNodeStakeOncePerEpoch();
    if (!operatorNodes[msg.sender].contains(nodeId)) {
        revert AvalancheL1Middleware__NodeNotFound(nodeId);
    }
    // ... existing stake validation ...
    _initializeValidatorStakeUpdate(msg.sender, validationID, stakeAmount);

    if (!_requireMinSecondaryCollateralClasses(0, msg.sender)) {
        revert AvalancheL1Middleware__InsufficientStake();
    }
}
```

M-2 Sequential weight updates on the same validator cause stake accounting drift

Description

When an operator performs two sequential weight updates on the same validator within the same epoch, a mismatch between the stake cache used during initialization and completion causes `operatorLockedStake` accounting to become incorrect. This results in operator stake becoming permanently locked and unusable for future validator operations.

The issue stems from how `nodeStakeCache` is updated: `completeValidatorWeightUpdate` writes to `nodeStakeCache[currentEpoch + 1][vid]` (next epoch), while `_initializeValidatorStakeUpdate` reads from `nodeStakeCache[currentEpoch][vid]` (current

epoch). When both operations happen in the same epoch, the completion of the first update never updates the current epoch's cache, causing the second initialization to read stale data.

Consider a validator with `nodeStakeCache[N][vid] = 100`:

1. First initiate (100→150): reads cache as 100, locks delta of 50
2. First complete: writes 150 to `nodeStakeCache[N+1]`, releases 50 (150-100)
3. Second initiate (150→200): reads stale cache as 100, locks delta of 100 instead of 50
4. Second complete (epoch N+1): reads correct 150 from `nodeStakeCache[N+1]`, releases only 50 (200-150)

The operator locked 150 total (50+100) but only released 100 (50+50), leaving 50 units of stake permanently orphaned in `operatorLockedStake`. This reduces the operator's available stake for future validator operations, and the drift accumulates with each sequential same-epoch update. There is no recovery mechanism since only `completeValidatorWeightUpdate` can decrease this value.

Recommendation

Modify `_initializeValidatorStakeUpdate` to prefer the next epoch's cached stake when available, since a non-zero value there indicates a weight update completed earlier in the same epoch:

```
function _initializeValidatorStakeUpdate(address operator, bytes32
validationID, uint256 newStake) internal {
    uint48 currentEpoch = getCurrentEpoch();
    uint256 cachedCurrent = getEffectiveNodeStake(currentEpoch,
validationID);
    uint256 cachedNext = getEffectiveNodeStake(currentEpoch + 1,
validationID);
    uint256 cachedStake = cachedNext > 0 ? cachedNext : cachedCurrent;

    if
(balancerValidatorManager.isValidatorPendingWeightUpdate(validationID)) {
        revert AvalancheL1Middleware__WeightUpdatePending(validationID);
    }
    ...
}
```

I-1 Missing constructor validation in UptimeTracker

Description

The `UptimeTracker` constructor accepts two parameters but performs no validation on either:

```
constructor(
    address payable middleware_,
```

```

        bytes32 uptimeBlockchainID_
    ) {
        middleware = AvalancheL1Middleware(middleware_);
        epochDuration = middleware.EPOCH_DURATION();
        validatorManager = BalancerValidatorManager(middleware.BALANCER());
        uptimeBlockchainID = uptimeBlockchainID_;
    }

```

If `middleware_` is `address(0)`, the constructor will revert with a low-level error when calling `middleware.EPOCH_DURATION()`. If `uptimeBlockchainID_` is `bytes32(0)`, the contract will deploy but all uptime messages will fail the source chain ID check.

This is inconsistent with the thorough validation in `RewardsNativeToken.initialize`, which validates all address parameters with specific error types:

```

if (admin_ == address(0)) revert InvalidAdmin(admin_);
if (protocolOwner_ == address(0)) revert
InvalidProtocolOwner(protocolOwner_);
if (middleware_ == address(0)) revert InvalidL1Middleware(middleware_);
if (uptimeTracker_ == address(0)) revert
InvalidUptimeTracker(uptimeTracker_);

```

Recommendation

Add validation checks to the constructor for consistency and clearer error messaging:

```

constructor(
    address payable middleware_,
    bytes32 uptimeBlockchainID_
) {
    if (middleware_ == address(0)) revert InvalidMiddleware();
    if (uptimeBlockchainID_ == bytes32(0)) revert InvalidBlockchainID();

    middleware = AvalancheL1Middleware(middleware_);
    epochDuration = middleware.EPOCH_DURATION();
    validatorManager = BalancerValidatorManager(middleware.BALANCER());
    uptimeBlockchainID = uptimeBlockchainID_;
}

```

I-2 Silent vault skipping in `_calculateAndStoreVaultShares` hinders debugging

Description

In `RewardsNativeToken._calculateAndStoreVaultShares`, several external calls are wrapped in `try/catch` blocks that silently skip the vault on failure without emitting any event or log:

```
// L785-787: delegator lookup
try IVaultTokenized(vault).delegator() returns (address d) {
    delegator = d;
} catch { continue; }

// L791-793: stake lookup
try BaseDelegator(delegator).stakeAt(
    middleware.BALANCER(), cls, operator, epochTs, new bytes(0)
) returns (uint256 s) { vaultStake = s; } catch { continue; }

// L800: operator stake lookup (sets to 0 instead of continuing)
try middleware.getOperatorStake(operator, epoch, cls) returns (uint256 s2) {
    sum = s2; } catch { sum = 0; }
```

While this defensive pattern prevents a single misbehaving vault or delegator from blocking reward distribution for all other participants, the silent failure makes it difficult to debug why a particular vault is not receiving rewards. Off-chain monitoring and troubleshooting become challenging when there's no on-chain indication that a vault was skipped or why.

Recommendation

Consider emitting an event when a vault is skipped due to a failed external call. This would aid debugging without affecting the control flow:

```
event VaultSkipped(uint48 indexed epoch, address indexed vault, string
reason);

// Example usage:
try IVaultTokenized(vault).delegator() returns (address d) {
    delegator = d;
} catch {
    emit VaultSkipped(epoch, vault, "delegator_call_failed");
    continue;
}
```

I-3 Duplicate code in LSTWrapper implementations

Description

`LSTWrapper` and `LSTWrapperMerk1` are nearly identical contracts (~600 lines each), differing only in how rewards are claimed via their respective harvest functions. `LSTWrapper` claims from a `RewardsNativeToken` contract, while `LSTWrapperMerk1` claims from a Merkle distributor using proofs.

All other functionality appears to be identical across both contracts. This duplication increases the maintenance burden and risk of divergence if one contract is updated but not the other.

Recommendation

Extract the shared logic into a common `LSTWrapperBase` contract and have both implementations inherit from it, overriding only the harvest mechanism.

I-4 `VAULT_REMOVAL_EPOCH_DELAY` lacks validation despite suggesting fixed value

Description

The `MiddlewareVaultManager` contract's `VAULT_REMOVAL_EPOCH_DELAY` constant has a comment suggesting it should be a fixed value, but it's set via constructor parameter without any validation to enforce expected bounds.

```
uint48 public immutable VAULT_REMOVAL_EPOCH_DELAY; // 1 epoch delay for
removal, affects rewards
```

Recommendation

Add constructor validation to ensure the delay falls within acceptable bounds.

I-5 Magic number 1 used instead of `PRIMARY_ASSET_CLASS` constant

Description

Several locations use the literal 1 instead of the `PRIMARY_ASSET_CLASS` named constant, inconsistent with the codebase's general practice of avoiding magic numbers.

Recommendation

Replace magic number 1 with `PRIMARY_ASSET_CLASS` for consistency and clarity.

I-6 Unnecessary unchecked loop increments

Description

Some loops use `unchecked { ++i }` for index increments. Since Solidity 0.8.22+, the compiler automatically optimizes loop increments when overflow is impossible. The codebase uses 0.8.25, making these manual unchecked blocks redundant.

Recommendation

Remove unnecessary `unchecked` blocks around loop increments.

I-7 `L1Registry.notZeroAddress` has overly specific parameter naming

Description

The `notZeroAddress` modifier in `L1Registry` uses a parameter named `l1`, but it's also used to validate middleware addresses. The parameter name and revert message are more specific than the actual usage.

Recommendation

Rename the parameter to a more generic name and update the revert message accordingly.

I-8 Misleading variable name in `RewardsNativeToken.claimRewards`

Description

The `maxClaimableEpoch` puts an upperbound on the number of epochs that can be claimed in one `claimRewards` call. However, it's calculated as `lastClaimedEpoch + 1 + MAX_EPOCHS_PER_CLAIM`, which is one higher than the actual maximum claimable epoch. The logic is correct because the loop uses `<` as the condition, but the naming suggests the value is the actual maximum claimable epoch rather than an exclusive upper bound.

Recommendation

Rename to `maxClaimableEpochPlusOne` or document the exclusive bound behavior clearly.

I-9 `LSTWrapper` and `LSTWrapperMerkle` `mint()` missing `maxMint` bypass for owner's first mint

Description

In both `LSTWrapper` and `LSTWrapperMerk1`, the `deposit()` function includes a special bypass for the owner's first mint that calls `_deposit()` directly, avoiding the `maxDeposit()` check in `super.deposit()`:

```
// For owner first mint, bypass maxDeposit check by calling _deposit directly
// Otherwise, use super.deposit() which checks maxDeposit()
if (isFirstMint && msg.sender == owner()) {
    _deposit(_msgSender(), receiver, assets, shares);
    return shares;
}
```

However, `mint()` lacks an equivalent bypass and always calls `super.mint()`. Since `maxMint()` returns `0` when `totalSupply() == 0` (to prevent non-owners from minting), the owner's first mint via `mint()` will always revert because OpenZeppelin's `ERC4626Upgradeable` enforces `shares <= maxMint(receiver)`.

Call flow for owner's first `mint()`:

1. `mint(shares, receiver)` passes the `isFirstMint && owner()` checks
2. Calls `super.mint(shares, receiver)`
3. `ERC4626Upgradeable` calls `maxMint(receiver)` which returns `0`
4. `ERC4626Upgradeable` enforces `require(shares <= 0) → reverts`

This means the owner can only seed the vault using `deposit()`, not `mint()`. While `deposit()` provides a working path for seeding, the `mint()` function also clearly intends to support first mint since it tracks `isFirstMint`, allows the owner to bypass the pause check, and restricts first mint to owner only. However, the necessary bypass to make it work is missing.

Recommendation

Add the same bypass pattern to `mint()` in both `LSTWrapper` and `LSTWrapperMerk1`:

```
function mint(uint256 shares, address receiver)
    public
    override(ERC4626Upgradeable, IERC4626)
    nonReentrant
    returns (uint256 assets)
{
    if (shares == 0) revert LSTWrapper__ZeroSharesMinted();

    LSTWrapperStorageStruct storage lws = _lstWrapperStorage();
    bool isFirstMint = totalSupply() == 0;
    if (lws.depositsPaused && !(isFirstMint && msg.sender == owner())) revert
    LSTWrapper__DepositsPaused();
    if (isFirstMint && msg.sender != owner()) revert
    LSTWrapper__OnlyOwnerFirstMint();

    // For owner first mint, bypass maxMint check by calling _deposit
    directly
```

```

    if (isFirstMint && msg.sender == owner()) {
        assets = previewMint(shares);
        _deposit(_msgSender(), receiver, assets, shares);
        return assets;
    }

    return super.mint(shares, receiver);
}

```

I-10 Unused `rewardsToken` parameter in `VaultHelper` reward functions

Description

Both `VaultHelper.getStakerClaimableReward` and `VaultHelper.getStakerClaimableRewardInRange` accept a `rewardsToken` parameter that is validated for non-zero but never actually used in the function logic:

```

function getStakerClaimableReward(
    address staker,
    address rewards,
    address vault,
    address rewardsToken // Validated but unused
) public view returns (ClaimAmountsPerToken memory) {
    if (staker == address(0)) revert VaultHelper__InvalidUser(staker);
    if (rewards == address(0)) revert VaultHelper__ZeroAddress("rewards");
    if (vault == address(0)) revert VaultHelper__ZeroAddress("vault");
    if (rewardsToken == address(0)) revert
VaultHelper__ZeroAddress("rewardsToken");
    // ...
    return ClaimAmountsPerToken({
        token: RewardsNativeToken(rewards).rewardsToken(), // Fetched from
contract, not parameter
        amount: totalAmount
    });
}

```

The actual rewards token address is fetched from `RewardsNativeToken(rewards).rewardsToken()` rather than using the provided parameter. This creates potential for confusion if a caller passes an incorrect `rewardsToken` address - the function will succeed but return data for a different token than the caller specified.

Recommendation

Either remove the `rewardsToken` parameter entirely, or validate that it matches the value from the rewards contract:

```
address actualRewardsToken = RewardsNativeToken(rewards).rewardsToken();
if (rewardsToken != actualRewardsToken) {
    revert VaultHelper__RewardsTokenMismatch(rewardsToken,
actualRewardsToken);
}
```

I-11 Duplicate code in VaultHelper functions

Description

`VaultHelper` contains several pairs of functions with significant code duplication:

1. **`stakeAssetInVault` / `stakeAssetInWrappedVault`**: Both functions share the same core flow (pull underlying tokens → deposit to collateral → deposit to vault), with the wrapped version adding extra steps to deposit vault shares into an LSTWrapper. The wrapped version also includes additional consistency validation (checking that vault collateral matches wrapper collateral, and collateral asset matches wrapper's native token) that the non-wrapped version lacks.
2. **`getUserPendingWithdraws` / `getUserPendingWithdrawsInRange`**: These functions have identical loop logic for counting and collecting pending withdrawals. The only difference is how the epoch range is determined: one uses a default range based on `MAX_EPOCH_SPAN`, while the other accepts explicit `fromEpoch/toEpoch` parameters.
3. **`getStakerClaimableReward` / `getStakerClaimableRewardInRange`**: These functions follow same pattern as above, having identical reward calculation loops, differing only in epoch range determination.

This is similar to the duplication noted in I-3 for the LSTWrapper implementations.

Recommendation

Extract shared logic into internal helper functions:

- For the stake functions, consider adding the consistency validation from `stakeAssetInWrappedVault` to `stakeAssetInVault` as well, then extract the common deposit flow into a shared internal function.
- For the lens function pairs, extract the loop logic into internal helpers that accept explicit epoch ranges, with the public functions computing the appropriate range before calling the helper.

I-12 Redundant `getAllOperators` call in

distributeRewards

Description

In `RewardsNativeToken.distributeRewards`, the operator set is queried twice via `middleware.getAllOperators()` within the same transaction:

```
// First call: auto-complete check (line 204)
bool fundingWindowClosedUnfunded = (currentEpoch > epoch +
FUNDING_DEADLINE_OFFSET) && !st.funded;
if (fundingWindowClosedUnfunded) {
    address[] memory opsNow = middleware.getAllOperators();
    if (opsNow.length == 0) {
        // auto-complete logic
    }
}

// Second call: snapshotting (line 215)
if (_epochOperators[epoch].length == 0) {
    _epochOperators[epoch] = middleware.getAllOperators();
}
```

Since both calls occur in the same transaction, they return identical results. The second call incurs unnecessary gas costs that scale linearly with the number of operators due to warm storage reads and memory allocation.

Recommendation

Move the snapshot logic before the auto-complete check and reuse the cached result:

```
// Snapshot first
if (_epochOperators[epoch].length == 0) {
    _epochOperators[epoch] = middleware.getAllOperators();
}

// Use snapshot for auto-complete check
bool fundingWindowClosedUnfunded = (currentEpoch > epoch +
FUNDING_DEADLINE_OFFSET) && !st.funded;
if (fundingWindowClosedUnfunded) {
    if (_epochOperators[epoch].length == 0) {
        distributionBatches[epoch].isComplete = true;
        st.distributionComplete = true;
        emit RewardsDistributed(epoch);
        return;
    }
}
```

}

}